

Machine Learning - II

Assignment - II

Submitted by

H K Shashank

23AM025

II - CSE (AIML)

Machine Learning - II

Name: HK Shashank

dept: III-CSE (AIML)

Rollno: 23AM025

Date: 22-10-2025

Assignment - II

1. Markov Decision Process

States (S)

Each intersection in the grid is a state. A state can be represented as

$$S = (x, y, z)$$

where

x, y : taxi's grid position

P : passenger status $\rightarrow$

0 - waiting

1 - onboard

2 - Delivered

Actions (A)

Possible actions at any intersection.

N - Move North

S - Move South

E - Move East

W - Move West

Pick - Pick up passenger

DROP - Drop off passenger

Transition Probabilities (P)

Probability of reaching from s after a :

$$P(s'|s, a) = \begin{cases} 1 - \epsilon & \text{if car move in successfully} \\ [\epsilon pt] \epsilon & \text{if taxi slip / stays in place} \\ [\epsilon pt] 0 & , \text{otherwise} \end{cases}$$

Reward function

Each step $\rightarrow -1$

Successful pickup $+10$

Successful drop $+100$

Collision -100

Illegal pickup -10

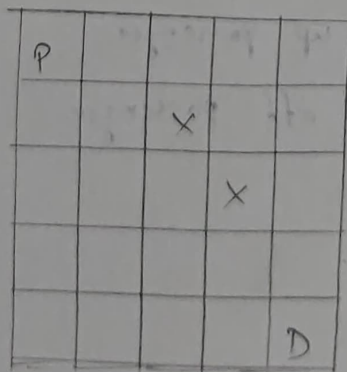
$$R(s, a, s') = \begin{cases} +100 & \text{if passenger dropped success} \\ -100 & \text{if collision occur} \\ -1 & \text{otherwise} \end{cases}$$

Objective function

Maximize expected discounted reward

$$\pi^* = \arg \max_{\pi} E_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \right]$$

Diagram.



~~Diagram~~ It is 5x5 grid diagram

Legend

P → Passenger Pickup point

D → Destination

X → Obstacle

Each intersection - state

Arrows represent - actions

Policy learning

To minimize travel time & avoid collisions we use

Q-learning

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R(s,a,s') + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

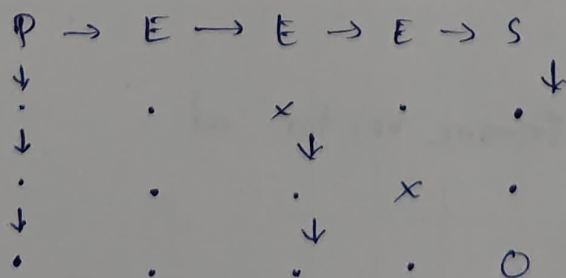
where, learning rate (0,1)

Discount factor (0,1)

Optimal policy is then $\pi^*(s) = \arg \max_a Q(s,a)$

Learning output example

After training learned policies might look like



when passenger onboard

* Follow path toward drop off

* Avoid obstacles

* Drop passenger at D

Real World Connection

Real Autonomous Taxi Equivalent

GPS Position, Passenger, Status

Movement decision

Trip efficiency

2. Hidden Markov Model

HMM formulation

HMM is defined by (S, O, A, B, π)

Hidden states (S)

discrete health states

$S = \{ \text{Calm, stressed, Anxious} \}$

Observation (O)

Continuous, sensor vector at

time t :

$$O_t = [HR_t, Temp_t, To hon_t]^T$$

Transition Matrix A :

$$a_{ij} = P(S_{t+1} = j \mid S_t = i)$$

Eg. $A = \begin{bmatrix} 0.90 & 0.80 & 0.02 \\ (+pt) 0.10 & 0.80 & 0.10 \\ (+pt) 0.05 & 0.20 & 0.75 \end{bmatrix}$

Emission Probabilities

for continuous observations, model,

$P(o_t \mid S_t = j)$ as a multivariate Gaussian

$$= N(o_t \mid \mu_j, \Sigma_j)$$

Initial Distribution

$$\pi = [0.8, 0.15, 0.05]$$

starts mostly calm

Inference - Forward & Backward

Initialization

$$S_i(j) = \log \pi_j + \log P(o_i \mid s_i = j)$$

Recursion

$$S_t(j) = \max_i [S_{t-1}(i) + \log a_{ij}] + \log P(o_t \mid s_t = j)$$

Posterior Marginals

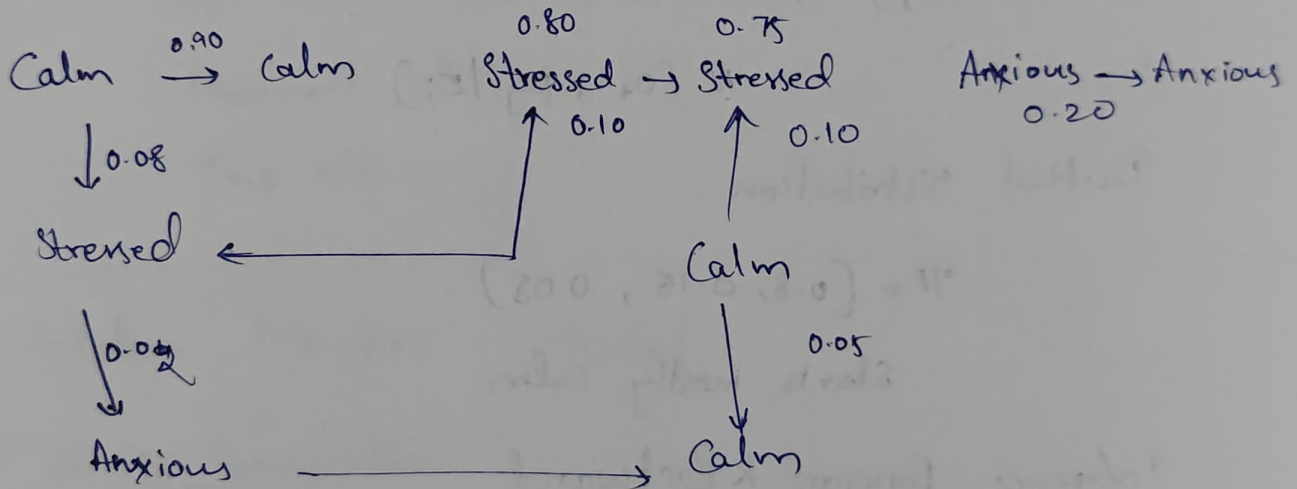
Computer α - Forward
 β - Backward

$$\alpha_i(j) = \pi_j P(o | s_i = j)$$

$$\beta_T(j) = 1$$

$$\gamma_t(j) = P(s_t = j | o_{1:T}) \propto \alpha_t(j) \beta_t(j)$$

8. State HMM Diagram



Code Snippet

```
# Health Monitoring
```

```
import numpy as np
```

```
# Hidden states
```

```
states = ["Calm", "Stressed", "Anxious"]
```

```
n = len(states);
```

Transition Probabilities

```
A = np.array([ [0.9, 0.08, 0.02],  
               [0.2, 0.6, 0.2],  
               [0.1, 0.3, 0.6] ])
```

initial state prob

```
Pi = np.array([0.7, 0.2, 0.1])
```

observation sequence

0 = Low HR, 1 = Medium HR, 2 = High R

```
O = [0, 1, 1, 2, 1, 0]
```

VITERBI ALG

```
T = len(O)
```

```
delta, np.zeros((T,n))
```

```
Psi = np.zeros((T,n),int)
```

initialization

```
delta[0] = np.log(Pi) + np.log([B[i, O[0]]])
```

Result

```
decoded = [state[i] for i in states_seq]
```

```
print("Most Likely hidden states")
```

```
print(decoded)
```


Output:

Most likely hidden states

['calm', 'stressed', 'stressed', 'Anxious']

3. Model Selection in HMM

Description

Daily market trends using as HMM, where hidden states represent latent market regimes and observations are market indicator such as

- * Daily return percentage
- * Trading volume
- * Volatile Index
- * Moving Average Signals

Why models selection matters

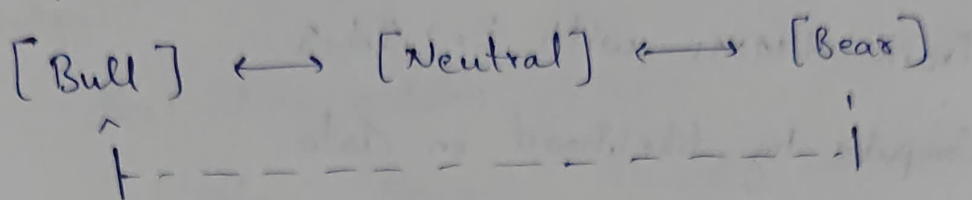
- Too few states $\rightarrow$

underfitting: model cannot capture all distinct market behaviours

- Too many states $\rightarrow$

overfitting: model memorises, noise instead of trends, reducing predictive generalization

Conceptual HMM



Each state emits observation

Bull $\rightarrow$ high returns
low volatility

Neutral $\rightarrow$ Small returns
moderate volume

Bear $\rightarrow$ negative return
high volume

Eval Candidate Models

Multiple HMM with different states counts
(eg $k = 2, 3, 4, 5$)

Eval using AIC, BIC, Cross-validator

Criterial for Model Selection

$$AIC = 2k - 2 \ln(2)$$

$$BIC = k \ln(N) - 2 \ln(2)$$

$$CV = \frac{1}{F} \sum_{i=1}^F (n \cdot D(O_{val}^{(i)} | \text{model } g_c))$$

Step-by-Step Procedure

Train HMMs with $K=2,3,4,5$

Compute log likelihood on data

Compute AIC & BIC for each K

Run 5-fold CV & Compute Average

Validation Score

Compare Results pick models

Intuitive Explanation

Increasing K (more states)

Improving training accuracy

But $\rightarrow$ model may invent states

Benefits

Detecting shifts help in portfolio rebalancing and risk control.

HMM model selection ensures robust performance prevents false alarms.

4. Q-Learning

Application Warehouse robot

States: Compact exam friendly state

(s) representation include

Robot grid position: (x, y) , $x, y \in \{0, 1, 2\}$

Carrying status: $c \in \{0, 1\}$

$$S = (x, y, c)$$

$$\text{no. of states} = 3 \times 3 \times 2 = 18$$

Actions (A)

Low-level discrete moves + task actions

$$A = \{\text{North, East, South, West, Pick, Drop}\}$$

Transition Dynamics

Pick succeeds only if robot at pickup cell &

$$C=0; \text{ then } C \rightarrow 1$$

Drop succeeds only if robot at drop cell

$C \rightarrow 0$, may end or continue for next task

Reward function

Step penalty (time / energy)

$$r_{\text{step}} = -1 \text{ per time step}$$

Movement energy cost

optionally add small additional negative reward per move proportional to distance

successful pickup = +0 or small +1

successful dropoff = +100

collision action = -10

battery penalty or long run penalty for energy

Can be modeled if required

Ex.

$$R(s, a, s') = \begin{cases} +100 & \text{successful drop} \\ -10 & \text{illegal drop} \\ -1 & \text{otherwise} \end{cases}$$

update rule

$$Q(s, a) \leftarrow Q(s, a) + \alpha (r + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

where

$Q(s, a)$ = action - value estimate

α = learning rate $\in [0, 1]$

γ = discount factor $\in [0, 1]$

Convergence & Parameter Choices

$0 < \alpha \leq 1$: Small but not too small

helps stability for guaranteed convergence
in some theory

γ : Choose near 1 if long term reward
is important, lower if immediate energy
matter more

Real world consideration

- * Add safety layers to block collision
use constrained RL for energy tradeoffs
- * Combine time & energy or use preth optimization
- * Train in simulator with domain randomization